

EECS 2311

Lab 5 (Take Home Assignment)

Rishab Yadav - 219618107

User Story	Main Developers	Member testing the implementation
Each Tab Functional (ITR1) Allow the user to swap between tabs during their session. These tabs do not have to be fully functional yet but being to swap between them must be. Priority Level: High, Cost: 4 days	Otokpa Attah Pietro Angiolella Rishab Yadav	Mohammad Waliduddin
Add User Authentication (ITR1) Enable users to create accounts, log in, and persist data across sessions. Priority Level: High, Cost: 3 days	Rishab Yadav	Pietro Angiolella
Store User Search History (ITR1) Store the user's search history, allowing users to revisit previous topics they've searched for. Priority Level: High, Cost: 2 days	Rishab Yadav Ondrea Peters	Pietro Angiolella
Display External Resources (ITR1) For each search, display at least 1 external resource or video. Priority Level: High, Cost: 3 days	Mohammad Waliduddin Rishab Yadav	Larissa Singh
Search Course Functional (ITR2) Be able to search a course and for it to grab the Rate Your Prof ratings for the next semester and a quick overview. Priority Level: High, Cost: 5 days	Rishab Yadav Otokpa Attah Larissa Singh Mohammad Waliduddin	Ondrea Peters
Refine Search Topic (ITR2) Refine the search process and roadmap generation. Limit google searches, better search options and be able to click on each Roadmap topic. Priority Level: High, Cost: 3 days	Pietro Angiolella Rishab Yadav	Otokpa Attah
Quiz Generation Possible (ITR2) Have a semi-working quiz generation tab. Will be refined in the coming iterations. Priority Level: Medium, Cost: 3 days	Pietro Angiolella	Rishab Yadav
Refined History (ITR2) Have a working history system with a favouriting system. Priority Level: Medium, Cost: 3 days	Pietro Angiolella Rishab Yadav	Rishab Yadav

End-to-End Manual Test Cases for Quiz Generation

Basic Quiz Generation Test Cases

Test Case QG-1: Generate Basic Quiz with Default Settings

Description: Verify that a user can generate a basic quiz with default settings **Priority:** High **Test Steps:**

1. Navigate to the quiz.html page
2. Click on "Generate Questions" card
3. Enter "Mathematics" in the topic input field
4. Leave default settings (Education Level: High School, Difficulty: Easy, Number of Questions: 10)
5. Click the "Generate" button
6. Wait for questions to load

Expected Results:

- The tool should generate 10 mathematics questions at high school level and easy difficulty
- Questions should be displayed in a grid format
- Each question box should have a "Show Answer" button

Test Case QG-2: Show Answer Functionality

Description: Verify that answers can be revealed and hidden for quiz questions **Priority:** High **Test Steps:**

1. Complete Test Case QG-1 to generate a quiz
2. Click the "Show Answer" button on the first question
3. Verify the answer appears
4. Click the "Hide Answer" button (previously "Show Answer")
5. Verify the answer disappears

Expected Results:

- When "Show Answer" is clicked, it should change to "Hide Answer"
- An answer should appear below the question
- When "Hide Answer" is clicked, the answer should be hidden
- Button text should change back to "Show Answer"

Test Case QG-3: Generate Quiz with Custom Settings

Description: Verify that a user can customize quiz settings **Priority:** High **Test Steps:**

1. Navigate to the quiz.html page
2. Click on "Generate Questions" card
3. Enter "Computer Science" in the topic input field
4. Change Education Level to "Year 3-4 University"
5. Change Difficulty to "Hard"
6. Change Number of Questions to 5
7. Click the "Generate" button
8. Wait for questions to load

Expected Results:

- The tool should generate 5 computer science questions
- Questions should be at a Year 3-4 University level
- Questions should be of Hard difficulty
- Badge on the quiz should display selected settings

Test Case QG-4: Generate Timed Test

Description: Verify that a timed test can be generated and timer functions correctly **Priority:** Medium **Test Steps:**

1. Navigate to the quiz.html page
2. Click on "Generate Test" card
3. Enter "Biology" in the topic input field
4. Set Time Limit to 5 minutes
5. Click the "Generate" button
6. Wait for test to load
7. Observe the timer counting down
8. Wait for timer to expire or click submit

Expected Results:

- A timed test should be generated with a 5-minute timer
- Timer should display and count down from 5:00
- Progress bar should update to reflect remaining time
- When timer reaches 0, a "Time's up!" message should appear

Test Case QG-5: Generate Flashcards

Description: Verify that flashcards can be generated **Priority:** Medium **Test Steps:**

1. Navigate to the quiz.html page
2. Click on "Generate Flashcards" card
3. Enter "Chemistry" in the topic input field
4. Set Number of Cards to 8
5. Choose Education Level "Year 1-2 University"
6. Click the "Generate" button
7. Wait for flashcards to load
8. Click on a flashcard to flip it

Expected Results:

- 8 chemistry flashcards should be generated
- Flashcards should be appropriate for Year 1-2 University level
- Clicking on a flashcard should flip it to show the answer
- Clicking again should flip it back to show the question

Edge Case Quiz Generation Test Cases

Test Case QG-6: Generate Quiz with Special Characters in Topic

Description: Verify the system handles topics containing special characters **Priority:** Low **Test Steps:**

1. Navigate to the quiz.html page
2. Click on "Generate Questions" card
3. Enter "C++ Programming" in the topic input field
4. Click the "Generate" button
5. Wait for questions to load

Expected Results:

- The tool should handle the special characters in the topic
- Questions about C++ Programming should be generated
- No errors should be displayed

Test Case QG-7: Generate Quiz with Very Long Topic Name

Description: Test if the system can handle very long topic names **Priority:** Low **Test Steps:**

1. Navigate to the quiz.html page
2. Click on "Generate Questions" card
3. Enter "The Comprehensive Analysis of Quantum Mechanics and Its Applications in Modern Physics and Technology Development" in the topic input field
4. Click the "Generate" button
5. Wait for questions to load

Expected Results:

- The system should process the long topic name
- Appropriate questions should be generated
- UI should properly handle the display of the long topic

Test Case QG-8: Generate Quiz with Minimum Settings

Description: Test generating quiz with minimum possible settings **Priority:** Medium **Test Steps:**

1. Navigate to the quiz.html page
2. Click on "Generate Questions" card
3. Enter "Math" in the topic input field
4. Set Number of Questions to 1
5. Click the "Generate" button
6. Wait for question to load

Expected Results:

- Only 1 question should be generated
- Question should be displayed properly
- "Show Answer" functionality should work correctly

Test Case QG-9: Generate Quiz with Maximum Settings

Description: Test generating quiz with maximum allowed settings **Priority:** Medium **Test Steps:**

1. Navigate to the quiz.html page
2. Click on "Generate Questions" card
3. Enter "Physics" in the topic input field
4. Set Number of Questions to 20 (maximum)
5. Click the "Generate" button

6. Wait for questions to load

Expected Results:

- 20 questions should be generated
- All questions should be displayed properly in the grid
- Page should be scrollable to view all questions

Test Case QG-10: Empty Topic Input

Description: Verify system prevents generation with empty topic **Priority:** Medium **Test Steps:**

1. Navigate to the quiz.html page
2. Click on "Generate Questions" card
3. Leave the topic input field empty
4. Click the "Generate" button

Expected Results:

- System should display an alert saying "Please enter a topic"
- No questions should be generated
- User should be able to enter a topic after seeing the alert

End-to-End Manual Test Cases for History Functionality

Basic History Functionality Test Cases

Test Case HF-1: View Search History

Description: Verify that search history is visible on the history page **Priority:** High **Test Steps:**

1. Perform a search on the search-topic.html page
2. Navigate to history.html
3. Observe the search history tiles

Expected Results:

- The search performed should be visible in the history
- Search tile should show query, education level, and timestamp
- If no searches have been performed, "No search history found" message should be displayed

Test Case HF-2: Delete a Search from History

Description: Verify that a single search can be deleted from history **Priority:** High **Test Steps:**

1. Navigate to history.html
2. Find a search entry to delete
3. Click the delete button (trash icon) on the search entry
4. Confirm deletion when prompted

Expected Results:

- Confirmation dialog should appear
- After confirming, the search should be removed from history
- Success message "Search deleted successfully!" should be shown
- If it was the last search, "No search history found" message should appear

Test Case HF-3: Create a Folder

Description: Verify folders can be created to organize searches **Priority:** High **Test Steps:**

1. Navigate to history.html
2. Click "Create Folder" button
3. Enter "Study Materials" as folder name
4. Enter "My study materials collection" as description
5. Click "Create Folder" button

Expected Results:

- New folder should be created with provided name and description
- Folder should appear in the folders container
- Success message should be displayed

Test Case HF-4: Move Search to Folder

Description: Verify searches can be moved to folders using drag and drop **Priority:** High **Test Steps:**

1. Navigate to history.html

2. Create a folder if one doesn't exist (Test Case HF-3)
3. Drag a search tile to the folder
4. Observe the result

Expected Results:

- Search should be removed from main history view
- "Item moved to folder successfully!" message should appear
- When viewing the folder, the search should be inside it

Test Case HF-5: Open a Folder and View Contents

Description: Verify folders can be opened to view their contents **Priority:** High **Test Steps:**

1. Navigate to history.html
2. Create a folder if one doesn't exist (Test Case HF-3)
3. Move a search to the folder (Test Case HF-4)
4. Click on the folder to open it

Expected Results:

- Folder view should open
- Folder name and description should be displayed
- Search items inside the folder should be visible
- Back button should be available

Test Case HF-6: Move Search Out of Folder

Description: Verify searches can be moved out of folders back to main history **Priority:** Medium **Test Steps:**

1. Navigate to history.html
2. Open a folder containing search items (Test Case HF-5)
3. Drag a search item to the back button
4. Observe the result

Expected Results:

- Search should be removed from folder
- Search should appear in main history view
- "Item moved out of folder successfully!" message should appear

Test Case HF-7: Delete a Folder

Description: Verify that folders can be deleted **Priority:** Medium **Test Steps:**

1. Navigate to history.html
2. Create a folder if one doesn't exist (Test Case HF-3)
3. Click the delete button (trash icon) on the folder
4. Confirm deletion when prompted

Expected Results:

- Warning dialog should mention that contents will be deleted too
- After confirming, folder and its contents should be removed
- "Folder and its contents deleted successfully!" message should appear

Test Case HF-8: Change Folder Color

Description: Verify folder color can be changed **Priority:** Low **Test Steps:**

1. Navigate to history.html
2. Create a folder if one doesn't exist (Test Case HF-3)
3. Click the color palette icon on the folder
4. Select a different color (e.g., blue)

Expected Results:

- Color menu should appear when clicking the palette icon
- Selecting a color should change the folder's color
- "Folder color updated!" message should appear

Test Case HF-9: Delete All History

Description: Verify all history and folders can be deleted at once **Priority:** Medium **Test Steps:**

1. Navigate to history.html
2. Ensure there are search entries and folders
3. Click "Delete All History" button
4. Confirm deletion when prompted

Expected Results:

- Warning dialog should appear with clear message about consequences
- After confirming, all searches and folders should be removed

- "All history and folders deleted successfully!" message should appear
- "No search history found" message should be displayed

Edge Case History Functionality Test Cases

Test Case HF-10: Create Folder with Maximum Length Description

Description: Verify the system handles maximum length folder descriptions **Priority:** Low **Test Steps:**

1. Navigate to history.html
2. Click "Create Folder" button
3. Enter "Test Folder" as folder name
4. Enter a 50-character description (maximum allowed)
5. Click "Create Folder" button

Expected Results:

- Folder should be created with the full description
- Character counter should show "50/50 characters"
- No errors should occur

Test Case HF-11: Create Folder with Same Name

Description: Verify the system handles multiple folders with the same name **Priority:** Low **Test Steps:**

1. Navigate to history.html
2. Create a folder named "Duplicate"
3. Create another folder with the same name "Duplicate"

Expected Results:

- Both folders should be created successfully
- System should distinguish between them (by creation time or internal ID)
- No errors should occur

Test Case HF-12: Open Empty Folder

Description: Verify empty folders display appropriate message **Priority:** Medium **Test Steps:**

1. Navigate to history.html
2. Create a new folder
3. Click on the folder to open it without adding any items

Expected Results:

- Folder view should open
- "No items in this folder" message should be displayed
- Folder empty state icon should be visible

Test Case HF-13: Open Search from History

Description: Verify searches can be reopened from history **Priority:** High **Test Steps:**

1. Navigate to history.html
2. Find a search entry
3. Click the open search button (external link icon)

Expected Results:

- User should be redirected to search-topic.html
- The search query should be prefilled
- Search settings (education level, roadmap steps) should be restored

Test Case HF-14: Move Multiple Searches to Folder

Description: Verify multiple searches can be moved to a folder **Priority:** Medium **Test Steps:**

1. Navigate to history.html
2. Create a folder if one doesn't exist
3. Drag one search item to the folder
4. Drag another search item to the same folder
5. Open the folder

Expected Results:

- Both search items should be removed from main history
- Both search items should appear in the folder
- Success messages should appear for each move

Test Case HF-15: Quick Dark Mode Toggle While in Folder View

Description: Verify dark mode toggle works correctly while in folder view **Priority:** Low **Test Steps:**

1. Navigate to history.html
2. Open a folder
3. Toggle dark mode using the button in the navigation bar
4. Return to main history view

Expected Results:

- Dark mode should toggle in folder view
- UI elements should maintain proper visibility
- When returning to main view, dark mode setting should persist

Bugs and Code Smells Report

Quiz Generation Bugs

1. **Hardcoded API Key Exposure (Security Bug)**
 - The Gemini API key is hardcoded directly in the JavaScript file: `const apiKey = 'AlzaSy/QLYD2wHZ49VgCichAxZXBWFI0BJA'`
 - **Impact:** This is a significant security vulnerability as anyone can extract the key and use it without restrictions.
2. **Inconsistent Quiz Count Limitation**
 - In `genQuiz()`, there's a list of 8 questions: `count = 8; //` but
 - But the UI allows selecting up to 20 questions (from `quiz.html`).
 - **Impact:** Users can select 20 questions but will only receive 8 at most.
3. **Unused Parameter in `genAns()`**
 - The `detailLevel` variable is calculated from difficulty but never used in the prompt.
 - **Impact:** The difficulty setting doesn't actually affect answer detail level as it should.
4. **Question Parsing Inconsistency**
 - The question parsing in `formatQuestions()` assumes specific formatting from the API response.
 - If Gemini returns questions without numbers or in an unexpected format, parsing could fail.
 - **Impact:** Some valid responses might not be correctly displayed as separate questions.
5. **Race Condition in `showAns()`**
 - If a user clicks "Show Answer" multiple times rapidly, multiple API calls could be triggered.
 - **Impact:** Could lead to UI glitches or displaying wrong answers for questions.

History Functionality Bugs

1. **Firestore Version Inconsistency**
 - `History.htm` uses Firestore v11.2.0 modules, but `test_history.js` uses the older namespace-based Firestore.
 - **Impact:** Could lead to compatibility issues if both are used together.
2. **Missing Error Handling in Folder Operations**
 - Most folder operations (like deleting folders) don't have complete error handling for UI feedback.
 - **Impact:** Users might not know if an operation failed or why.
3. **Potential Memory Leak with Event Listeners**
 - In `draggedTile`, event listeners are added but not always properly removed.
 - **Impact:** Could cause memory leaks with prolonged use.
4. **Incomplete XSS Protection**
 - User input (like folder names) is directly inserted into the DOM without proper sanitization.
 - **Impact:** Could allow cross-site scripting attacks if users add malicious content.
5. **Race Condition in Drag and Drop**
 - Async operations during drag and drop could lead to inconsistent state if multiple operations happen quickly.
 - **Impact:** Items might appear in wrong locations or duplicate after drag operations.

Code/Design Smells

Checked for the following code smells and found these issues:

1. **Duplicate Code**
 - **Education Level and Difficulty Mapping:** The same switch statements for mapping education levels and difficulty are duplicated in both `genQuiz()` and `genAns()` functions.
 - **Folder Creation Logic:** Similar code for creating and appending folders appears in multiple places.
2. **Long Functions**
 - `createFolderElement()`: This function is over 100 lines long and handles multiple responsibilities.
 - `formatQuestions()`: This function is doing too much – parsing, formatting, and building HTML.
3. **Magic Numbers/Strings**
 - **Hard-coded values:** Numbers like 8 (max questions) and times like 3000 (ms for status messages) are not explained.
 - **CSS class names:** Strings like `text-warning` are scattered throughout the code without context.
4. **Global State**

- storedAnswers object: Used to cache answers globally, potentially causing conflicts.
 - draggedTile variable: Tracks the currently dragged tile globally, which could lead to state issues.
5. **Inadequate Error Handling**
- Most API calls and database operations have minimal error handling.
 - User-facing error messages are generic and don't provide useful information.
6. **Tight Coupling**
- (No specific details provided in the image, but the report indicates it exists.)